

Regression — Normal Equation

Use this when the exam says: "*use the normal equation*", or doesn't specify a method (normal equation is the default for regression).

How it works: Set gradient of MSE = 0, solve algebraically. One shot — no iterations, no learning rate.

$$W = (\Phi^T \Phi)^{-1} \Phi^T t$$

Case	Design matrix Φ row
Linear (1 var)	<code>[1, x]</code>
Polynomial degree d	<code>[1, x, x², ..., x^d]</code>
Multivariate (n vars)	<code>[1, x1, x2, ..., xn]</code>

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
```

Case 1 — Linear Regression (1 variable)

```
In [ ]: # --- dummy data ---
np.random.seed(0)
x_train = np.linspace(0, 10, 50)
t_train = 3*x_train + 2 + np.random.randn(50)*3
m = x_train.shape[0]
# -----

# Design matrix: each row = [1, x]
Phi = np.column_stack([np.ones(m), x_train]) # shape (m, 2)
print('Phi shape:', Phi.shape)
print('First row:', Phi[0]) # [1.0, x[0]]
```

```
In [ ]: # Normal equation
W = np.linalg.inv(Phi.T @ Phi) @ Phi.T @ t_train
print('W:', W.round(4)) # [intercept, slope]

t_pred = Phi @ W
rmse = np.sqrt(np.mean((t_train - t_pred)**2))
print(f'RMSE: {rmse:.4f}')
```

```
In [ ]: plt.scatter(x_train, t_train, color='steelblue', edgecolors='navy', s=40, label='Tr
plt.plot(x_train, t_pred, 'r-', linewidth=2.5, label='Linear fit')
plt.xlabel('x')
```

```
plt.ylabel('t')
plt.title(f'Linear Regression (Normal Equation) - RMSE={rmse:.3f}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Case 2 — Polynomial Regression (degree d)

```
In [ ]: # --- dummy quadratic data ---
np.random.seed(0)
x_train = np.linspace(-3, 3, 50)
t_train = 2*x_train**2 - x_train + 1 + np.random.randn(50)*2
m = x_train.shape[0]
# -----

DEGREE = 2 # change based on scatter plot shape

def build_design_matrix(x, degree):
    # Each row = [1, x, x^2, ..., x^degree]
    return np.column_stack([x**i for i in range(degree + 1)])

Phi = build_design_matrix(x_train, DEGREE) # shape (m, degree+1)
print('Phi shape:', Phi.shape)
print('First row:', Phi[0].round(4)) # [1, x, x^2]

In [ ]: # Normal equation - same formula regardless of degree
W = np.linalg.inv(Phi.T @ Phi) @ Phi.T @ t_train
print('W:', W.round(4)) # [w0, w1, w2, ...]

t_pred = Phi @ W
sigma_pred = np.sqrt(np.mean((t_train - t_pred)**2)) # RMSE = expected uncertainty
print(f'RMSE (expected uncertainty): ±{sigma_pred:.4f}')

In [ ]: x_line = np.linspace(x_train.min()-0.1, x_train.max()+0.1, 300)
t_line = build_design_matrix(x_line, DEGREE) @ W

plt.figure(figsize=(9, 5))
plt.fill_between(x_line, t_line-sigma_pred, t_line+sigma_pred,
                 alpha=0.2, color='red', label=f'±1σ = ±{sigma_pred:.2f}')
plt.fill_between(x_line, t_line-2*sigma_pred, t_line+2*sigma_pred,
                 alpha=0.1, color='red', label='±2σ (95%)')
plt.scatter(x_train, t_train, color='steelblue', edgecolors='navy',
            s=40, alpha=0.8, zorder=5, label='Training data')
plt.plot(x_line, t_line, 'r-', linewidth=2.5, zorder=4, label=f'Degree-{DEGREE} fit')
plt.xlabel('x1')
plt.ylabel('t')
plt.title(f'Polynomial Regression degree {DEGREE} (Normal Equation) - RMSE={sigma_p
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Case 3 — Multivariate Linear Regression (n variables)

Same Normal Equation. Design matrix just has more columns: $[1, x_1, x_2, \dots, x_n]$.

Note: normalize features first so all features are on the same scale.

```
In [ ]: # --- dummy multivariate data (2 features) ---
np.random.seed(0)
X_train = np.column_stack([np.random.randn(47)*500 + 2000, # feature 1: house size
                           np.random.randint(1, 6, 47)]) # feature 2: num room
t_train = 100*X_train[:, 0] + 5000*X_train[:, 1] + np.random.randn(47)*10000
m, n = X_train.shape
print('X_train:', X_train.shape) # (47, 2)
print('t_train:', t_train.shape) # (47,)
```

```
In [ ]: # Normalize (important – features have very different scales)
mean = X_train.mean(axis=0) # (n,)
std = X_train.std(axis=0) # (n,)
X_norm = (X_train - mean) / std

# Add bias column
Phi = np.hstack([np.ones((m, 1)), X_norm]) # shape (m, n+1)
print('Phi shape:', Phi.shape)
```

```
In [ ]: # Normal equation – exactly the same formula
W = np.linalg.inv(Phi.T @ Phi) @ Phi.T @ t_train
print('W:', W.round(2)) # [w0(bias), w1, w2, ...]

t_pred = Phi @ W
rmse = np.sqrt(np.mean((t_train - t_pred)**2))
print(f'RMSE: {rmse:.2f}')
```

```
In [ ]: # Predict a new sample – MUST normalize with training mean and std
x_new = np.array([1650, 3]) # raw values
x_new_norm = (x_new - mean) / std # normalize with TRAINING std
x_new_bias = np.append(1, x_new_norm) # add bias → [1, x1_norm, x2_norm]
t_new_pred = x_new_bias @ W
print(f'Prediction for {x_new}: {t_new_pred:.2f}')
```

Summary

```
# Linear (1 var)
Phi = np.column_stack([np.ones(m), x]) # (m, 2)

# Polynomial degree d
Phi = np.column_stack([x**i for i in range(d+1)]) # (m, d+1)
```

```

# Multivariate (normalize first, then add bias)
X_norm = (X - mean) / std
Phi     = np.hstack([np.ones((m, 1)), X_norm])           # (m, n+1)

# Normal equation – same for all three cases
W       = np.linalg.inv(Phi.T @ Phi) @ Phi.T @ t
t_pred  = Phi @ W
sigma_pred = np.sqrt(np.mean((t - t_pred)**2))           # RMSE

```